

UNIT-I

Introduction to Operating Systems: Types (Mainframe, Desktop, Real-Time, Embedded, Cloud, Mobile OS). OS Structures: Kernel, Shell, System Calls. CPU Scheduling: Concepts and Algorithms (FCFS, SJF, RR, Priority).

Introduction to Operating Systems:

An operating system (OS) is a program that acts as an **intermediary** between a computer user and the computer hardware. Its goals are to execute user programs, make solving user problems easier, make the computer convenient to use, and use the hardware in an efficient manner.

Example:

- **Shared Systems (e.g., Mainframes):** Must maximize **resource utilization**—ensuring that all available CPU cycles, memory, and I/O are explicitly balanced to keep all users happy.
- **Dedicated Systems (e.g., Workstations/Desktops):** Optimized primarily for **user convenience, ease of use, and high performance**, completely disregarding resource utilization since the hardware resources are dedicated to a single individual.
- **Mobile Devices:** Highly resource-poor; must optimize strictly for **usability and battery life**.
- **Embedded Systems:** Run primarily without user intervention, often lacking any user interface (UI) entirely.

Types of Operating System:

1. Mainframe Systems & Batch Environments

- **Context & Focus:** These represent the early evolutionary lineage of modern operating systems, designed to manage high-volume, centralized computational tasks.

- **Core Characteristics:**

- **Batch Systems:** The OS processes tasks in batches. A user does not interact directly with the computer system; instead, they prepare a job (consisting of the program, data, and control information) and submit it to a computer operator.
- **Multiprogramming (Crucial Step):** To eliminate massive CPU idle time caused by slow I/O devices, multiprogramming organizes jobs so the CPU always has one to execute. A subset of total jobs is kept simultaneously in memory. When the current job must wait for an I/O operation, the OS switches the CPU to another job.
- **Time-Sharing (Multitasking):** An extension of multiprogramming where the CPU executes multiple jobs by switching among them so frequently that users can interact with each program while it is running, creating an interactive computing environment.
- **Examples**
 - z/OS
 - z/VM

2. Desktop Systems (Workstations & PCs)

- **Context & Focus:** Evolved alongside microprocessors to provide dedicated personal computing environments.
- **Core Characteristics:**
 - The primary architectural objective shifts away from maximizing CPU utilization toward **user convenience and responsiveness**.
 - These systems natively support robust Graphical User Interfaces (GUIs) alongside traditional Command-Line Interfaces (CLIs).
 - While they began as single-user platforms without advanced memory protection, modern desktop OS adopt full multiprogramming, multitasking, and advanced virtual memory architectures originally refined in larger systems.
 - **Examples**
 - Microsoft Windows 11
 - macOS
 - Ubuntu

3. Real-Time Operating Systems (RTOS)

- **Context & Focus:** Utilized when rigid time requirements have been placed on the operation of a processor or the flow of data.
- **Core Characteristics:**
 - **Deterministic Behavior:** A real-time system functions correctly only if it returns the exact correct computational results within strictly defined time constraints (deadlines).
 - **Hard Real-Time Systems:** Guarantee that critical tasks will be completed on time. Secondary storage is typically limited or absent, with data stored directly in short-term memory or Read-Only Memory (ROM).
 - **Soft Real-Time Systems:** Provide less stringent scheduling. A critical real-time task receives absolute priority over non-critical tasks and retains that priority until it completes, though missing a deadline does not mean total system failure.
 - **Examples**
 - VxWorks
 - QNX
 - FreeRTOS

4. Embedded Systems

- **Context & Focus:** These systems are embedded inside devices like automobiles, home appliances, industrial robots, and medical equipment.
- **Core Characteristics:**
 - They almost always run a highly specialized **Real-Time Operating System (RTOS)** underneath.
 - They are heavily resource-constrained, functioning with very limited memory, small chips, and absolute dependence on power efficiency.
 - Their functionality is entirely fixed and predetermined, executing dedicated software tasks without direct human intervention or standard desktop-style application layers.
 - **Examples**
 - Embedded Linux
 - Windows IoT

5. Cloud Systems & Distributed Environments

- **Context & Focus:** Built upon modern distributed systems where computing tasks are spread across multiple physically separate, networked systems.
- **Core Characteristics:**
 - **Virtualization Core:** Cloud environments leverage extensive virtualization, where the physical hardware runs a specialized hypervisor or host OS that spins up independent Virtual Machines (VMs), allowing guest operating systems to execute isolated workloads.
 - **Resource Elasticity:** The environment dynamically abstracts hardware resources (compute power, storage, network access) so they can be provisioned or scaled seamlessly on demand.
 - **Load Balancing & Distribution:** Heavy emphasis on network protocols and distributed storage systems to ensure high availability and prevent single points of failure across data centers.
 - **Examples**
 - OpenStack
 - VMware vSphere

6. Mobile Operating Systems

- **Context & Focus:** Built specifically to manage tablets and smartphones.
- **Core Characteristics:**
 - **Resource Disadvantages:** Unlike tethered desktops, mobile hardware is explicitly constrained by battery capacity, physical thermal limits, and smaller memory sizes.
 - **Middleware Frameworks:** To support application developers, mobile OS architecture relies heavily on middleware—a dedicated set of software frameworks that provide advanced features like multimedia support, graphics rendering engines, and localized database management.
 - **Alternative User Interfaces:** Differentiates itself by handling touch-screen inputs, gesture recognition, voice commands, and GPS location services natively inside its core I/O subsystems.
 - **Examples**
 - Android
 - iOS

OS Structures: (Kernel, Shell and System Calls):

An operating system provides an environment for the execution of programs by safeguarding hardware resources while providing user interfaces. The internal structure of an OS is divided into layers that separate user-facing operations from core hardware management.

1. The Shell (User Interface Layer)

Computers employ two primary architectural approaches for users to issue commands directly to the operating system: **Command-Line Interfaces (CLI)** and **Graphical User Interfaces (GUI)**.

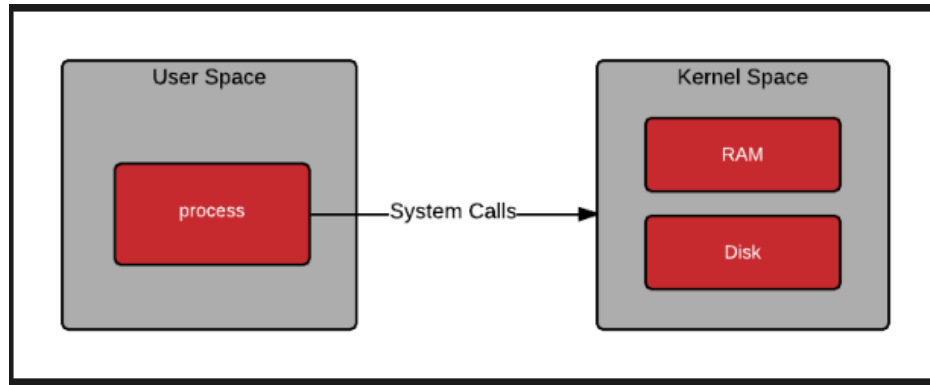
Command Interpreters (The Shell)

On systems that feature a text-based interface, the command interpreter is commonly referred to as the **shell**.

- **Core Function:** The shell acts as the outermost layer of the operating system structure. It reads commands directly from the user or from a batch file and executes them, typically by translating the command name into an underlying system execution path.
- **Varieties:** On systems like Linux or Unix, multiple distinct shells are available for choice (e.g., the Bourne-Again Shell or bash, csh, tcsh). These exist fundamentally as system programs outside the core operating system code itself.
- **Execution Strategies:** Command interpreters implement one of two operational strategies when executing user requests:
 1. *Internal Code Execution:* The shell itself contains the precise code to execute the command (e.g., a simple file deletion command managed internally by the interpreter).
 2. *System Program Invocations:* The shell uses the text command to locate a standalone file in storage, loads that file into memory, and transfers control to it.

2. The Kernel (Core Layer)

The **kernel** as the foundational core program that runs at all times on the computer. Everything else is classified cleanly as either a system program or an application program.



Core Responsibilities of the Kernel

The kernel acts simultaneously as a resource allocator and a control program across these operational areas:

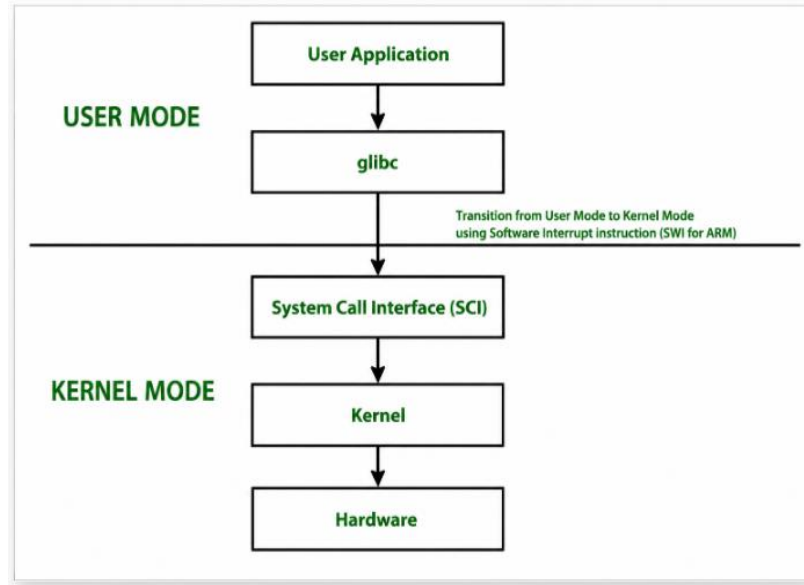
- **Process Management:** Allocates CPU execution time, creates/destroys processes, and provides inter-process communication (IPC) frameworks.
- **Memory Management:** Tracks which parts of primary memory are currently in use and by whom, while dynamically allocating and deallocating memory space as programs execute.
- **Storage & File System Management:** Manages files and directories, maps logical storage concepts onto physical block devices (HDDs, SSDs), and governs access permissions.
- **I/O Subsystem Management:** Buffers, caches, and handles device driver abstractions to shield application code from physical hardware variations.

The System Call Interface: Connecting Shell & Kernel

User programs and shells cannot access physical hardware directly because doing so risks total system crashes. When a shell or application needs a privileged resource (e.g., reading a file from storage), it must issue a **System Call**.

OPERATING SYSTEMS

To safeguard the system against malicious or malfunctioning programs, modern hardware implements a mode bit providing dual-mode execution i.e. (User mode and Kernel mode) as shown below:



- **User Mode (Mode Bit = 1):** The execution mode of standard application programs and shells. Privileged hardware instructions are completely blocked.
- **Kernel Mode (Mode Bit = 0):** The execution mode when the operating system kernel is performing tasks. Access to all physical memory locations and hardware controllers is fully authorized.

Functions of System Calls:

1. File System Operations

These system calls are made while working with files in OS, File manipulation operations such as creation, deletion, termination etc.

- **open():** Opens a file for reading or writing (text, audio, etc.).
- **read():** Reads data from an opened file.
- **write():** Writes or saves data to a file.
- **close():** Closes an opened file.
- **seek():** Moves the file pointer to a specific position in a file (e.g., jump to line 47 to read from there).

2. Process Control

These types of system calls deal with process creation, process termination, process allocation, deallocation etc. Basically manages all the process that is a part of OS.

- **fork():** Creates a new child process by duplicating the parent process.
- **exec():** Replaces the current process with a new program (overlay execution).
- **wait():** Makes the parent process wait until its child process finishes.
- **exit():** Terminates the current process.
- **kill():** Sends a signal to a process (e.g., terminate, stop, or restart).

3. Memory Management

These types of system calls deals with memory allocation, deallocation & dynamically changing the size of a memory allocated to a process. In short, the overall management of memory is done by making these system calls.

- **brk():** Sets the end address of the heap, directly resizing heap memory.
- **sbrk():** Adjusts the heap size by a positive (increase) or negative (decrease) value.
- **mmap():** Maps a file or device into a process's memory space so it can be accessed like normal memory.
- **munmap():** Removes a memory mapping from a process's address space.
- **mlock() / munlock():** Locks pages in memory to prevent swapping; unlock releases them back to normal management.

4. Interprocess Communication (IPC)

When two or more process are required to communicate, then various IPC mechanism are used by the OS which involves making numerous system calls. Some of them are :

- **pipe():** Creates a one-way communication channel between processes (e.g., parent → child).
- **socket():** Creates a network socket for communication between processes on the same or different systems.
- **shmget():** Allocates a shared memory segment so multiple processes can access the same memory space.
- **semget():** Creates or accesses a semaphore, used for process synchronization when sharing resources.

- **msgget():** Creates or accesses a message queue, allowing processes to exchange messages in a structured way.

5. Device Management

The device management system calls are used to interact with various peripheral devices attached to the PC or even the management of the current device.

- **SetConsoleMode():** Sets the input/output mode of the console (e.g., controls command-line behavior in Windows).
- **WriteConsole():** Writes data to the console screen.
- **ReadConsole():** Reads data from the console input.
- **open():** Opens a file or device and returns a file descriptor for access.
- **close():** Closes a previously opened file or device.

CPU Scheduling:

1. Introduction

In a multi programmed system, several processes reside in main memory at the same time. Whenever one process must wait (typically for I/O), the operating system takes the CPU away from it and gives the CPU to another process. This simple idea — always keeping the CPU busy with some process — is the essence of CPU scheduling, and it is central to making efficient use of a computer's resources.

Scheduling of this kind is a fundamental operating-system function: almost all computer resources are scheduled before use, and the CPU is one of the primary computer resources.

2. Basic Concepts

2.1 The CPU–I/O Burst Cycle

Process execution consists of a cycle of CPU execution and I/O wait. Processes alternate between these two states:

- CPU burst – a period during which the process uses the CPU for computation.
- I/O burst – a period during which the process waits for an I/O operation to complete.

A process execution begins with a CPU burst, followed by an I/O burst, and then another CPU burst, and so on; the final CPU burst ends with a request to terminate execution. CPU-burst durations vary greatly from process to process and from computer to computer, but they tend to follow a frequency curve that is generally characterized as exponential or hyper exponential, with a large number of short bursts and a small number of long bursts.

An I/O-bound program typically has many short CPU bursts, while a CPU-bound program might have a few long CPU bursts. This distribution is important in the selection of an appropriate CPU-scheduling algorithm.

2.2 The CPU Scheduler

Whenever the CPU becomes idle, the operating system must select a process from the ready queue to be executed. The selection process is carried out by the short-term scheduler (or CPU scheduler). The scheduler selects a process from among the processes in memory that are ready to execute and allocates the CPU to that process.

Note that the ready queue is not necessarily a first-in, first-out (FIFO) queue; it may be implemented as a FIFO queue, a priority queue, a tree, or simply an unordered linked list, depending on the scheduling algorithm used.

2.3 Preemptive and Non-preemptive Scheduling

CPU-scheduling decisions may take place under the following four circumstances:

- 1. When a process switches from the running state to the waiting state (e.g., as a result of an I/O request).
- 2. When a process switches from the running state to the ready state (e.g., when an interrupt occurs).
- 3. When a process switches from the waiting state to the ready state (e.g., at completion of I/O).
- 4. When a process terminates.

When scheduling takes place only under circumstances 1 and 4, the scheduling scheme is non-preemptive (or cooperative). Otherwise, it is preemptive. Under

non-preemptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases it either by terminating or by switching to the waiting state. Under preemptive scheduling, a running process may be interrupted and moved to the ready state, allowing another process to run.

Preemptive scheduling can result in race conditions when data are shared among several processes, and requires careful design to ensure the consistency of shared data — this is why kernel data structures must often be protected via mechanisms such as locks during updates.

2.4 The Dispatcher

Another component involved in the CPU-scheduling function is the dispatcher — the module that gives control of the CPU to the process selected by the short-term scheduler. This function involves:

- Switching context from one process to another (a context switch).
- Switching to user mode.
- Jumping to the proper location in the user program to resume that program.

The dispatcher should be as fast as possible, since it is invoked during every process switch. The time it takes for the dispatcher to stop one process and start another running is known as the dispatch latency.

3. Scheduling Criteria

Different CPU-scheduling algorithms have different properties, and the choice of a particular algorithm may favor one class of processes over another. Many criteria have been suggested for comparing scheduling algorithms:

- CPU utilization — we want to keep the CPU as busy as possible.
- Throughput — the number of processes completed per unit time.
- Turnaround time — the interval from the time of submission of a process to the time of completion. It is the sum of the periods spent waiting to get into memory, waiting in the ready queue, executing on the CPU, and doing I/O.

- Waiting time — the sum of the periods spent waiting in the ready queue. It is not affected by the amount of time a process actually executes or does I/O — only by how long it waits in the ready queue.
- Response time — the time from submission of a request until the first response is produced (for interactive systems); the time it takes to start responding, not the time it takes to output the response.

It is desirable to maximize CPU utilization and throughput and to minimize turnaround time, waiting time, and response time. In most cases, the average measure is optimized; but in some circumstances it is the minimum or maximum value that matters, rather than the average — for example, minimizing the maximum response time to guarantee fairness.

4. Scheduling Algorithms

CPU scheduling deals with the problem of deciding which of the processes in the ready queue is to be allocated the CPU. The following sections describe four classic scheduling algorithms: First-Come, First-Served (FCFS); Shortest-Job-First (SJF); Round Robin (RR); and Priority Scheduling.

4.1 First-Come, First-Served (FCFS) Scheduling

First-Come, First-Served (FCFS) is the simplest CPU scheduling algorithm in which the process that arrives first in the ready queue is allocated the CPU first. All other processes wait in the queue until the currently executing process completes.

It follows the FIFO (First-In, First-Out) principle, just like customers waiting in a ticket counter or supermarket queue.

Key Characteristics

Type :	Non-preemptive scheduling algorithm.
Scheduling Rule:	Execute processes in the order of arrival.
Data Structure Used:	Queue (FIFO).
CPU Allocation:	Once a process gets the CPU, it continues execution until completion or voluntary I/O request.

OPERATING SYSTEMS

Example (arrival order P1, P2, P3, P4)

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	5
P2	1	3
P3	2	8
P4	3	6

Step 1: Order of Execution

Since FCFS schedules based on arrival time,

P1 → P2 → P3 → P4

Step 2: Draw the Gantt Chart

0 5 8 16 22
| P1 | P2 | P3 | P4 |

Step 3: Completion Time (CT)

Process	Completion Time
P1	5
P2	8
P3	16
P4	22

Step 4: Turnaround Time (TAT)

Formula

Turnaround Time = Completion Time – Arrival Time

Process	CT	AT	TAT = CT – AT
P1	5	0	5
P2	8	1	7
P3	16	2	14
P4	22	3	19

Step 5: Waiting Time (WT)

Formula

$$\text{Waiting Time} = \text{Turnaround Time} - \text{Burst Time}$$

Process	TAT	BT	WT
P1	5	5	0
P2	7	3	4
P3	14	8	6
P4	19	6	13

Final Result

Process	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	16	14	6
P4	3	6	22	19	13

Average Waiting Time = 5.75 ms

Average Turnaround Time = 11.25 ms

The Convoy Effect

The average waiting time under an FCFS policy is often quite long and depends heavily on the arrival order of processes. If a CPU-bound process arrives before several short I/O-bound processes, the short processes — and the I/O devices they would use — must wait for the long process to release the CPU. This results in lower CPU and device utilization than might be possible if the shorter processes were allowed to go first; this phenomenon is known as the convoy effect.

4.2 Shortest-Job-First (SJF) Scheduling

Shortest-Job-First (SJF) is a CPU scheduling algorithm in which the process having the smallest CPU burst time is selected for execution next.

If two processes have the same burst time, the one that arrived earlier is executed first.

SJF is designed to minimize the average waiting time among all scheduling algorithms.

Characteristics

- **Type:** Non-Preemptive
- **Selection Criterion:** Shortest CPU Burst Time
- **Goal:** Minimize Average Waiting Time
- **Data Structure:** Ready Queue sorted by Burst Time

Example (arrival order P1, P2, P3, P4)

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	5
P2	1	3
P3	2	8
P4	3	6

Execution Order

P1 → P2 → P4 → P3

Gantt Chart

0 5 8 14 22
| P1 | P2 | P4 | P3 |

Final Table

Process	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	1	3	8	7	4
P4	3	6	14	11	5
P3	2	8	22	20	12

Average Waiting Time = $21/4 = 5.25$ ms

Average Turnaround Time = $43/4 = 10.75$ ms

4.3 Priority Scheduling

Priority Scheduling is a CPU scheduling algorithm in which each process is assigned a priority value, and the CPU is allocated to the process with the highest priority.

In many operating systems, a smaller priority number indicates a higher priority (e.g., Priority 1 is higher than Priority 2).

If two processes have the same priority, FCFS (First-Come, First-Served) is used to break the tie.

Types of Priority Scheduling

1. Non-Preemptive Priority Scheduling

- Once a process is assigned the CPU, it continues execution until it finishes.
- The scheduler chooses the highest-priority process only when the CPU becomes free.

2. Preemptive Priority Scheduling

- If a newly arrived process has a higher priority than the currently running process, the CPU is immediately allocated to the new process.

Characteristics

- **Type:** Non-Preemptive (in this example)
- **Scheduling Criterion:** Highest Priority
- **Tie Breaking:** FCFS
- **Suitable for:** Real-time and critical applications

Example

Using the same processes, assign the following priorities.

Note: Lower priority number means higher priority.

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P1	0	5	2
P2	1	3	1
P3	2	8	4
P4	3	6	3

Execution Order

P1 → P2 → P4 → P3

Gantt Chart

0 5 8 14 22
| P1 | P2 | P4 | P3 |

Final Result

Process	AT	BT	Priority	CT	TAT	WT
P1	0	5	2	5	5	0
P2	1	3	1	8	7	4
P4	3	6	3	14	11	5
P3	2	8	4	22	20	12

Average Waiting Time = $21/4 = 5.25$ ms

Average Turnaround Time = $43/4 = 10.75$ ms

Note: In this particular example, the execution order of **Priority Scheduling** is the same as **SJF** because the assigned priorities happen to follow the same order as the burst times after **P1** completes. In general, **Priority Scheduling and SJF are different algorithms**, and they often produce different execution orders.

Starvation and Aging

A major problem with priority-scheduling algorithms is indefinite blocking (or starvation): a low-priority process may wait indefinitely for the CPU if there is a steady stream of higher-priority processes. A solution to this problem is aging — a technique of gradually increasing the priority of processes that wait in the system for a long time. For example, if priorities range from 127 (low) to 0 (high), the priority of a waiting process could be incremented periodically (for instance, every 15 minutes); even a process with an initial priority of 127 would eventually attain the highest priority and execute.

4.4 Round-Robin (RR) Scheduling

Round Robin (RR) is a **preemptive CPU scheduling algorithm** in which each process is allocated the CPU for a fixed amount of time called the **Time Quantum (Time Slice)**. If a process does not complete within its time quantum, it is **preempted** and moved to the end of the ready queue. The CPU is then assigned to the next process in the queue.

Round Robin ensures that every process gets an equal opportunity to execute, making it one of the most widely used scheduling algorithms in **time-sharing and interactive operating systems**.

Characteristics

- **Type:** Preemptive
- **Scheduling Rule:** FIFO order with a fixed time quantum.
- **Data Structure:** Circular Queue (Ready Queue)
- **Fairness:** Every process receives an equal CPU time slice.
- **Suitable for:** Time-sharing systems.

Key Term

Time Quantum (Time Slice)

A **Time Quantum (TQ)** is the maximum amount of CPU time allocated to a process before it is preempted.

For this example,

Time Quantum (TQ) = 2 ms

Example

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	5
P2	1	3
P3	2	8
P4	3	6

OPERATING SYSTEMS

Execution Sequence

P1 → P2 → P3 → P1 → P4 → P2 → P3 → P1 → P4 → P3 → P4 → P3

Gantt Chart

0 2 4 6 8 10 11 13 14 16 18 20 22
| P1 | P2 | P3 | P1 | P4 | P2 | P3 | P1 | P4 | P3 | P4 | P3 |

Final Result

Process	AT	BT	CT	TAT = CT - AT	WT = TAT - BT
P1	0	5	14	14	9
P2	1	3	11	10	7
P3	2	8	22	20	12
P4	3	6	20	17	11

Average Waiting Time = $39/4 = 9.75$ ms

Average Turnaround Time = $61/4 = 15.25$ ms

Comparison of Scheduling Algorithms

Algorithm	Execution Order	Average WT	Average TAT
FCFS	P1 → P2 → P3 → P4	5.75 ms	11.25 ms
SJF (Non-Preemptive)	P1 → P2 → P4 → P3	5.25 ms	10.75 ms
Priority (Non-Preemptive)	P1 → P2 → P4 → P3	5.25 ms	10.75 ms
Round Robin (TQ = 2 ms)	Multiple time slices	9.75 ms	15.25 ms
